Building SLMs From LLMs: Distilling, Shrinking, And Finetuning.

How to Make LLMs Smaller and Cheaper

- By Dr. Anand Jayaraman

© 2025 TekFrameworks

Recap of Day 1

- Embeddings
- Transformers
- BERT, GPT
- How LLMs like GPT5 are trained.

© 2025 TekFrameworks

What Is This Session About?

Three problems. Three solutions. One unified framework.

Problem: Wrong behavior
Needs to learn new tasks or style
→ **Fine-tuning / PEFT**

Your LLM

Problem: Need a smaller clone
Smaller model that thinks like a big one
→ **Distillation**

Problem: Too big / too slow
RAM / GPU / latency constraints
→ **Quantization**

These are not mutually exclusive — QLoRA combines Arrow 1 (PEFT) + Arrow 2 (Quantization)

© 2025 TekFrameworks

What Is The Need For Finetuning?

- In-context learning may not be able to include enough data samples to describe the problem well.
- In-context learning may not work for smaller LLMs
- Examples take up space in the context window.
- While LLM may be good with natural language understanding and generation, task can be specific.
  - Conversations
  - Code generation
  - Summarization
  - Sentiment analysis
- Finetuned small language models (SLMs) can perform very well.

© 2025 TekFrameworks

Why Fine-tune An LLM?

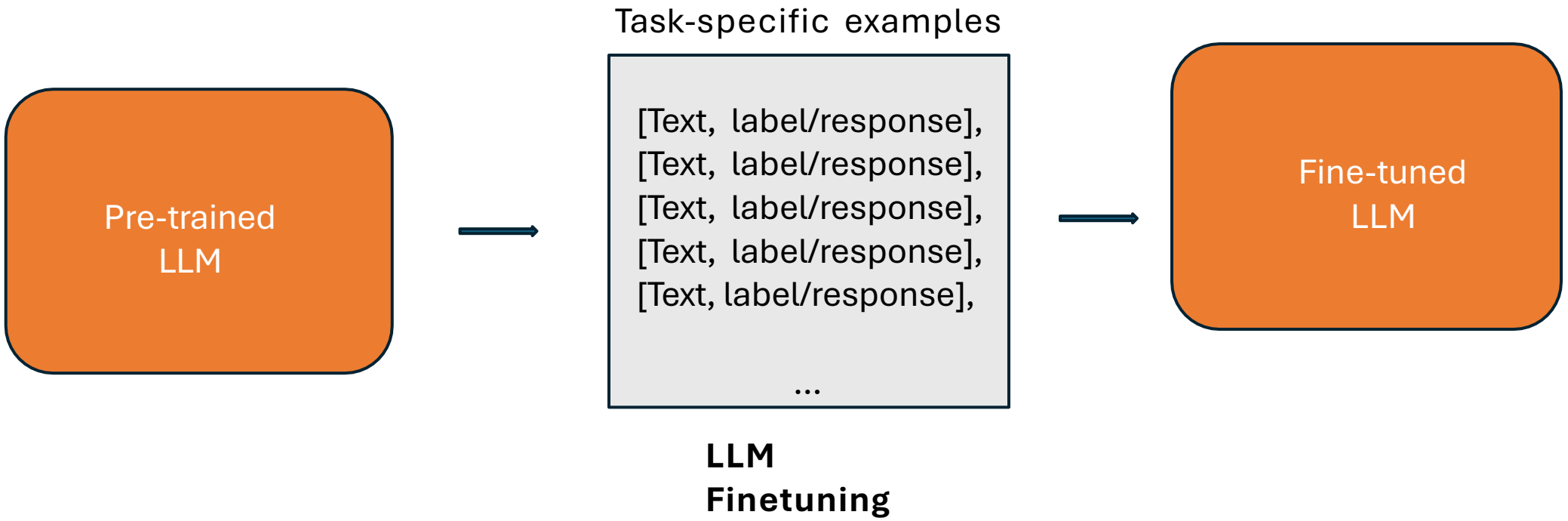
- Prompting = surface-level control
- Fine-tuning = embed knowledge or style into the model
- Prompting (temporary fix) vs Fine-Tuning (permanent learning)

Start with prompting.

*Use RAG if the model needs **external knowledge**.*
Use fine-tuning when the model itself needs to change how it behaves.

© 2025 TekFrameworks

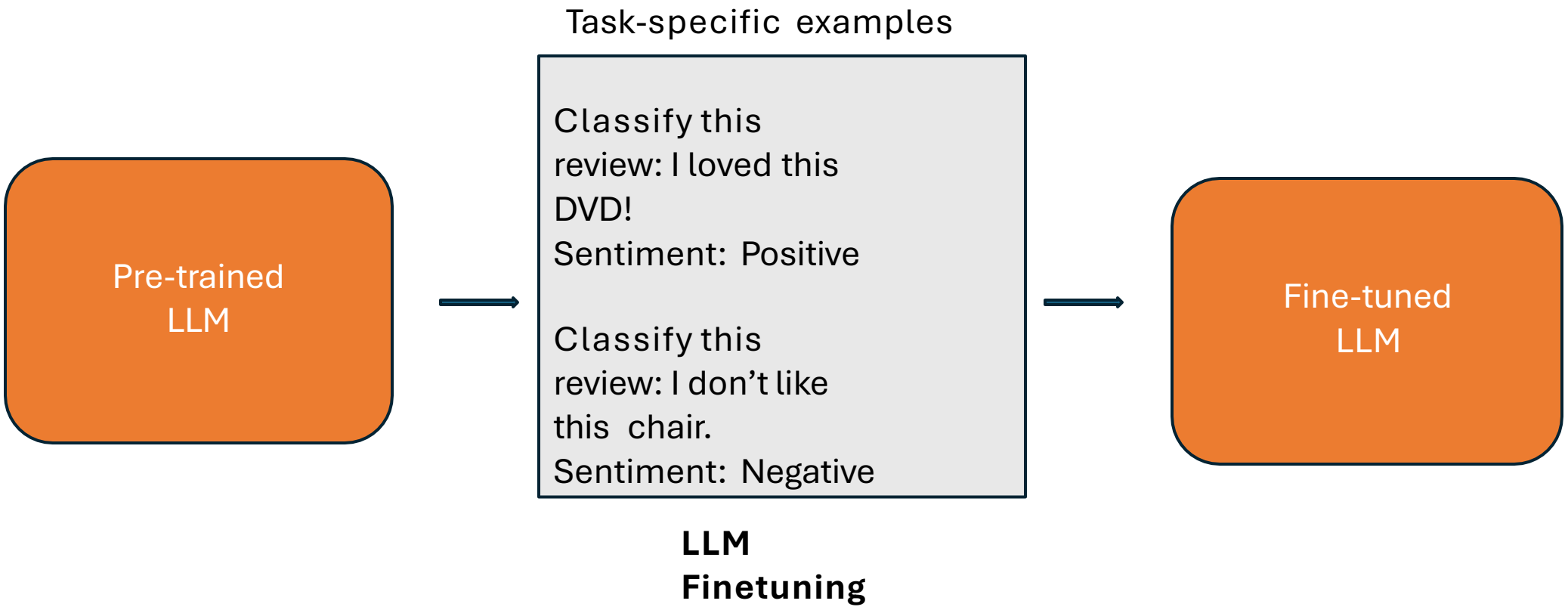
LLM Finetuning (High-level)



Pre-training already taught the model language and reasoning. Fine-tuning only needs to redirect it. So typically a few thousand examples are enough, not millions.

© 2025 TekFrameworks

Using prompts to fine-tune LLMs



Instruction fine-tuning in one line: The model learns to follow a task format, not just produce outputs — which is why fine-tuned models feel more "instruction-following" than base models.

© 2025 TekFrameworks

Prompt Templates For Fine-tuning

Summarize the following text: [EXAMPLE TEXT]

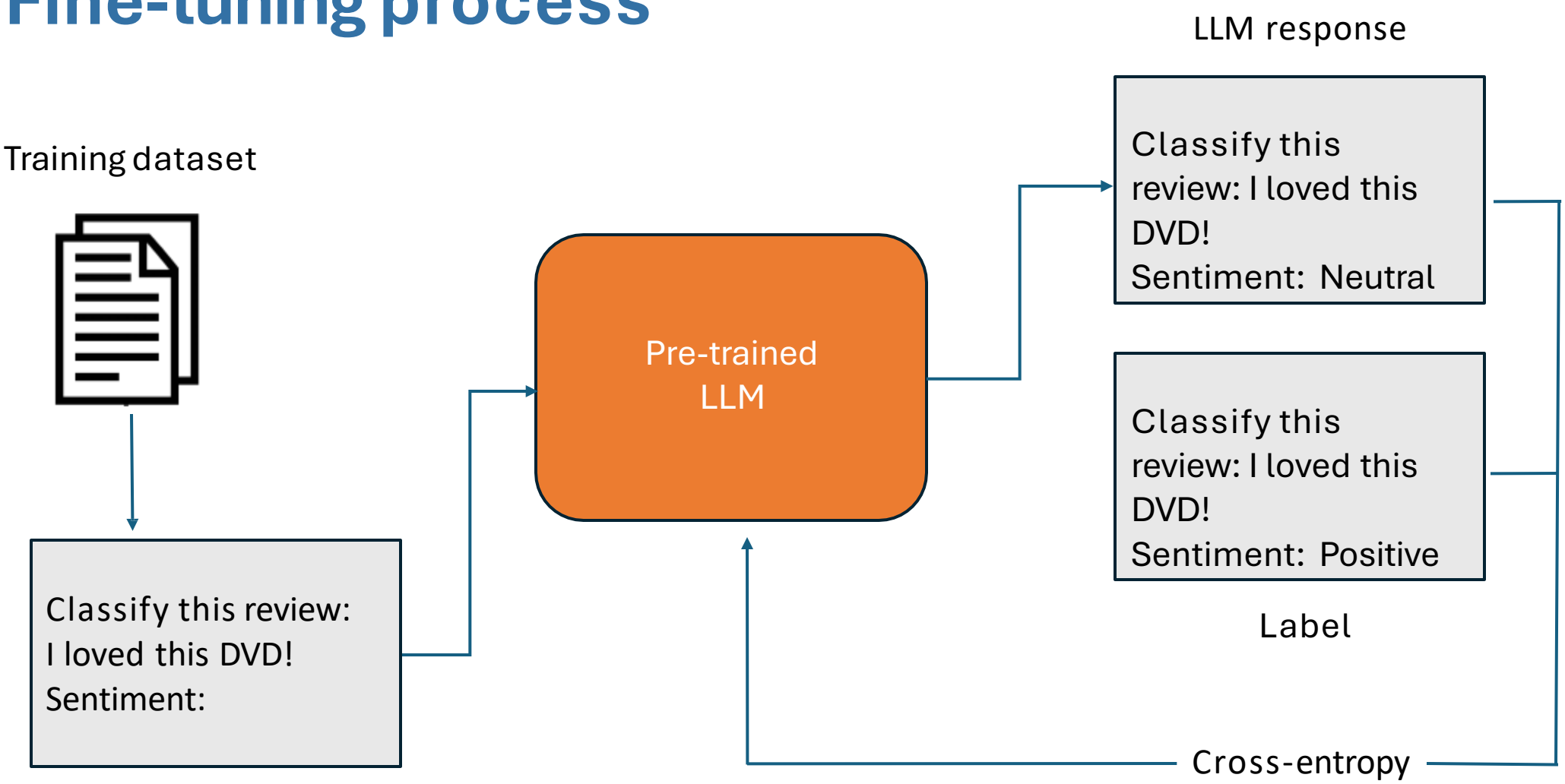
[EXAMPLE COMPLETION]

Translate this sentence to … [EXAMPLE TEXT]

[EXAMPLE COMPLETION]

© 2025 TekFrameworks

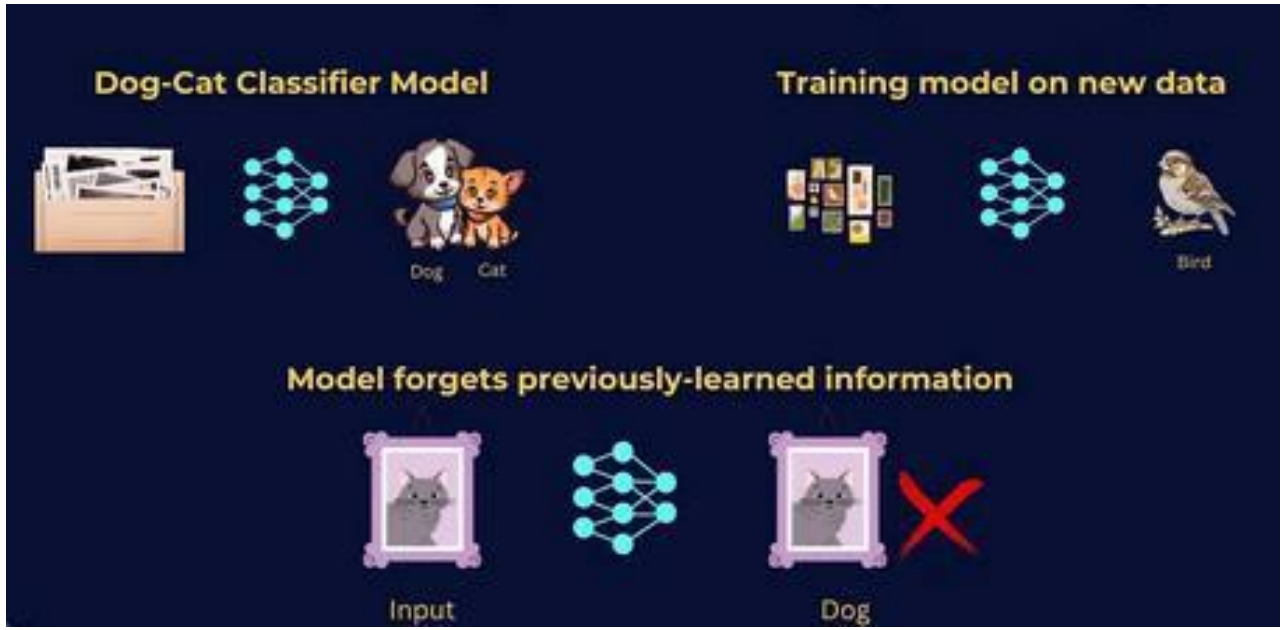
Fine-tuning process



© 2025 TekFrameworks

Tuning On A Single Task

- Only few thousand samples needed for finetuning on a task.
- Learning on a single task with less data can lead to reduction in ability on other tasks



Catastrophic forgetting can happen

© 2025 TekFrameworks

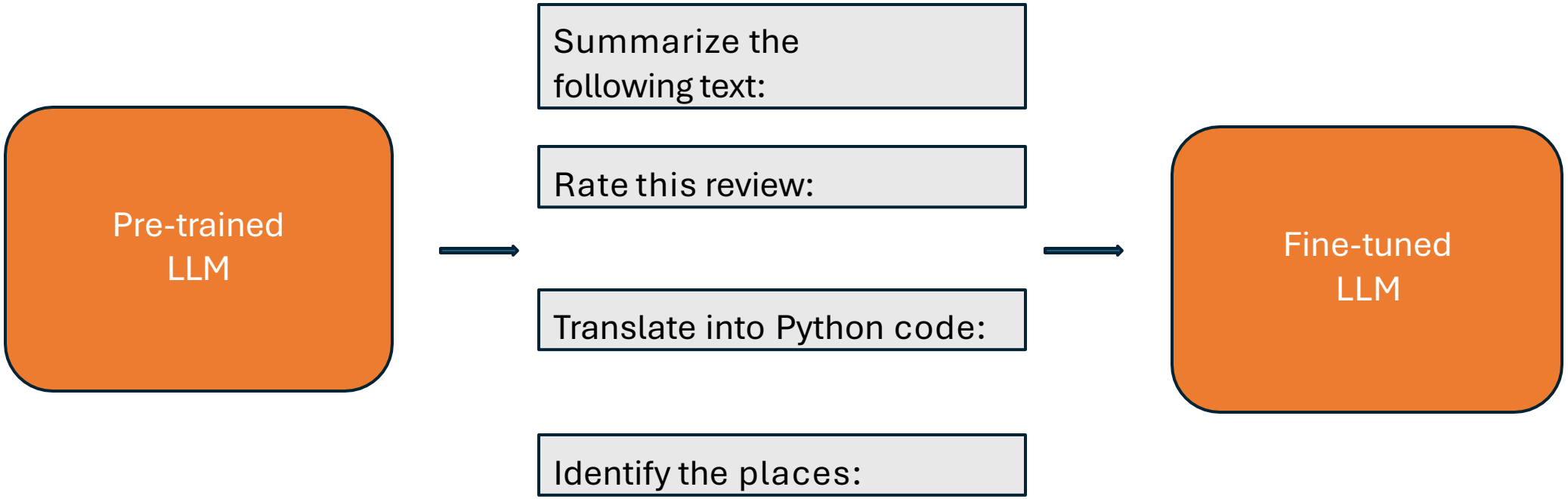
How To Avoid Catastrophic Forgetting

- It is not always necessary to avoid
- Accept forgetting when the deployment is single-task. Only solve it when you need multi-task capability.

Solution: Fine-tune on multiple tasks at the same time

© 2025 TekFrameworks

Multi-task finetuning of LLM



LLM Finetuning

© 2025 TekFrameworks

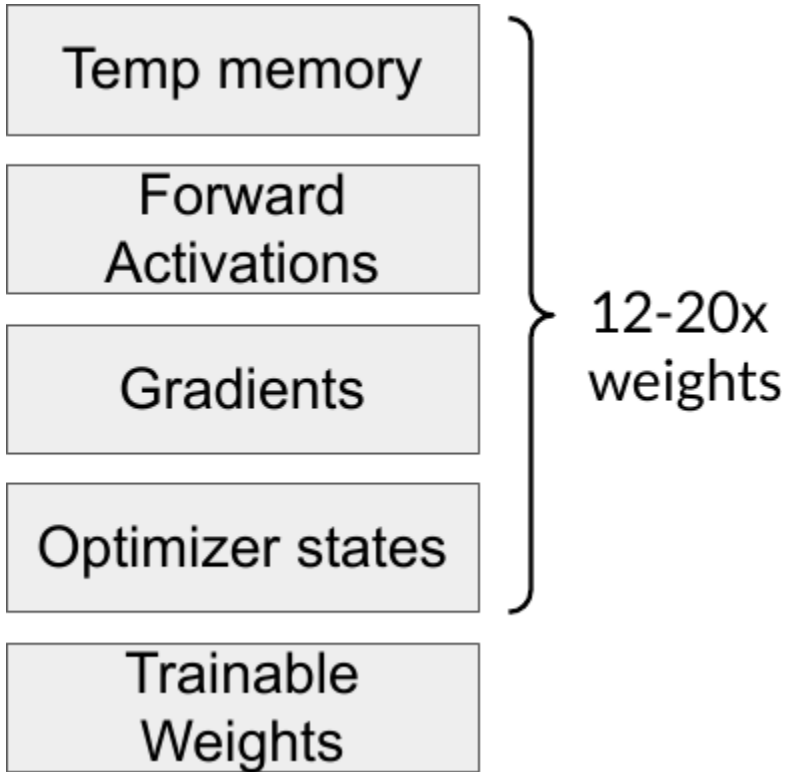
PEFT

→

© 2025 TekFrameworks

Full fine-tuning of large LLMs is challenging

- Full finetuning requires large number of computational resources.
- GPU memory requirement is equivalent to training the LLM from scratch.
- High budget implications.
- Need to create a large finetuned LLM per task or a group of tasks.



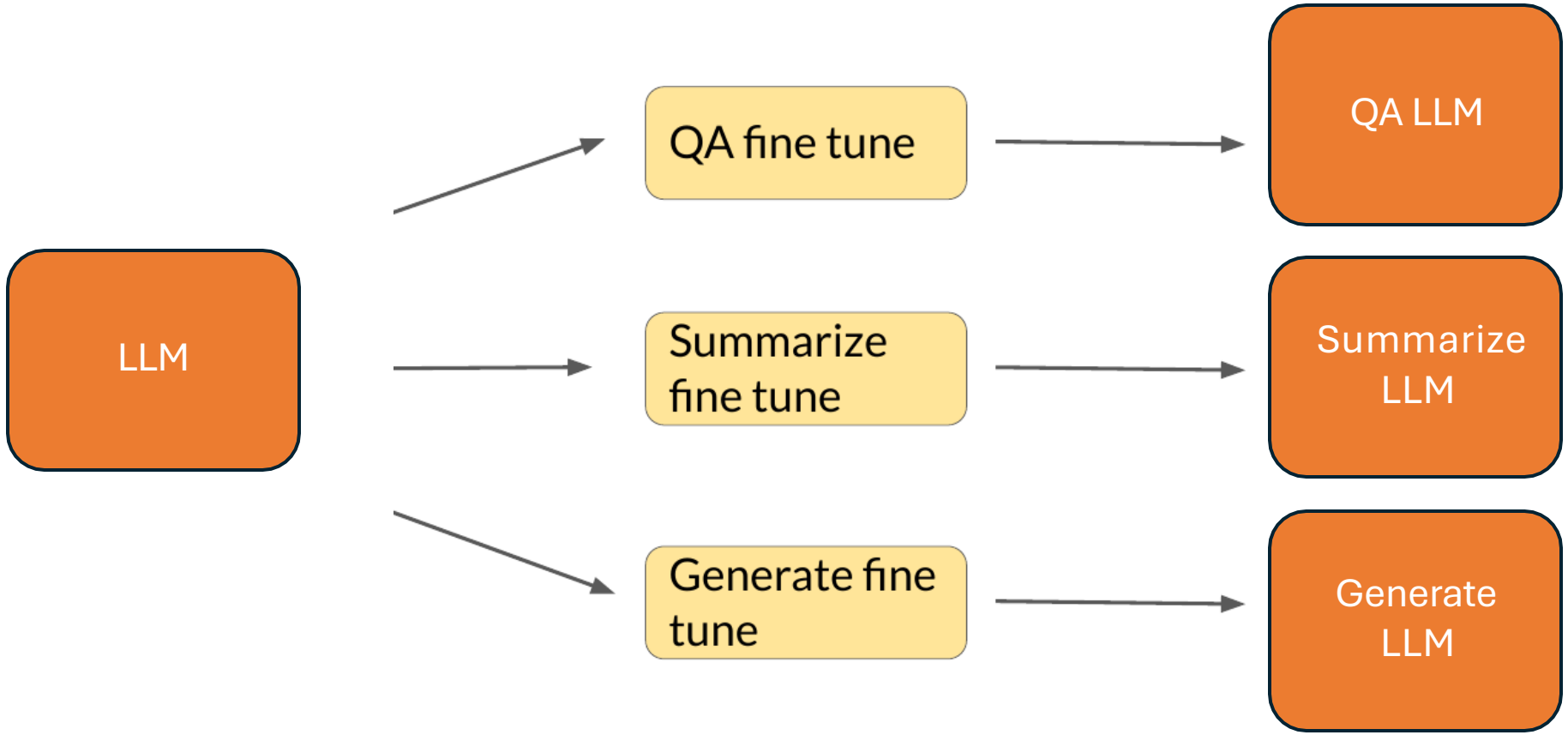
© 2025 TekFrameworks

Parameter Efficient Fine-tuning (PEFT)

- Only train a small set of additional parameters of the LLM
- Most layers of the LLM are frozen
- Less prone to catastrophic forgetting
- Less storage requirement
- Computationally less expensive
- Less budget requirement

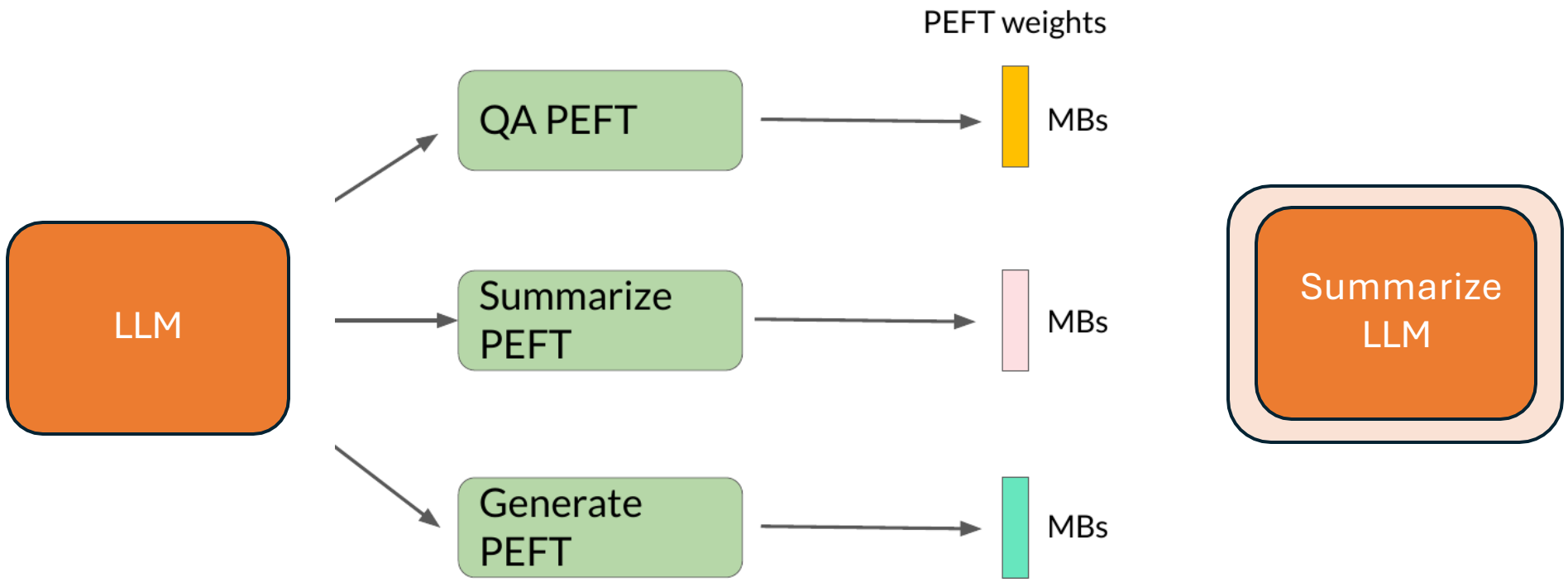
© 2025 TekFrameworks

Finetuning creates a full copy of the LLM



© 2025 TekFrameworks

PEFT finetuning saves cost and storage



© 2025 TekFrameworks

PEFT Methods

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

Adapters

Soft Prompts

Prompt Tuning

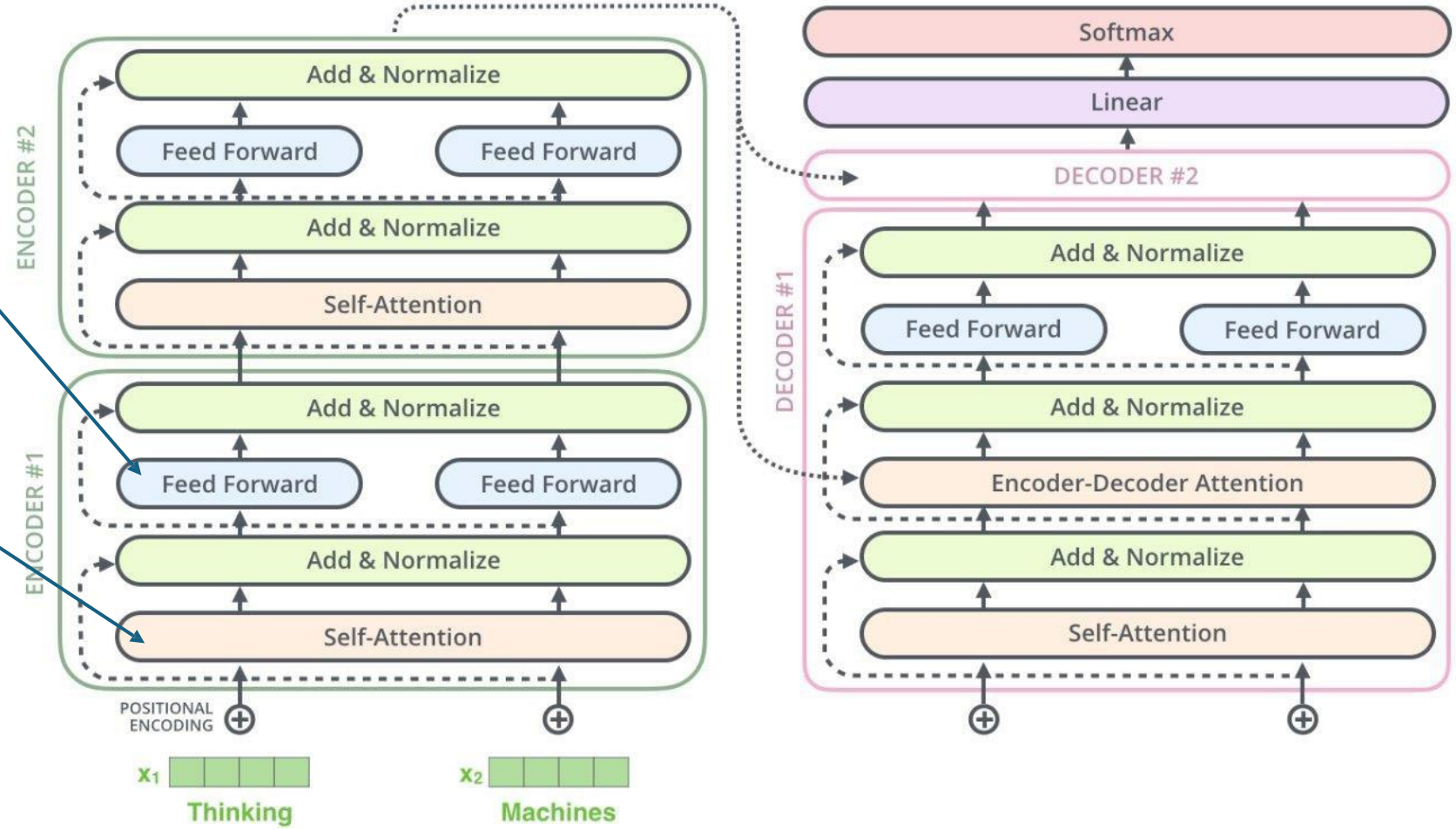
In practice, LoRA won. Adapters add sequential latency. Prompt tuning underperforms at small scales. LoRA adds zero inference overhead and trains efficiently.

© 2025 TekFrameworks

Transformers Recap

FCN weights

Attention weights



© 2025 TekFrameworks

Opening the Transformer Box: Two Families of Weights

TekFrameworks

Attention Weights

Capture how words relate to each other in context

- “bank” near “river” vs. “bank” near “money”
- Query, Key, Value projections (Q, K, V matrices)

LoRA targets these — freezes them and adds tiny trainable adapters alongside

Feed-Forward/Fully Connected Network (FCN) Weights

Store factual knowledge and reasoning patterns

- “Paris is the capital of France”
- Code syntax, grammar rules, domain knowledge

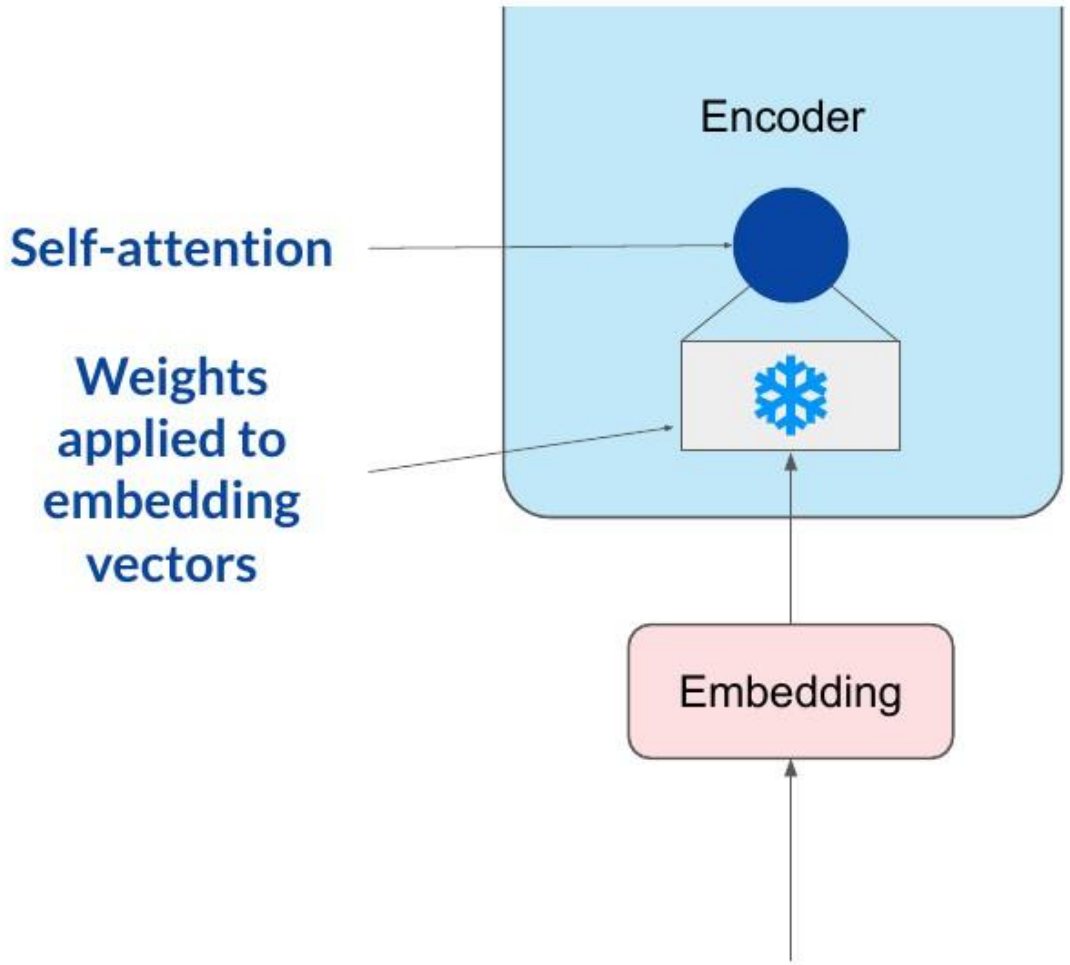
Also targeted in some LoRA configs for deeper task adaptation

Key insight: LoRA freezes both families and adds tiny trainable “adapters” — only 0.06% of all parameters

© 2025 TekFrameworks

LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

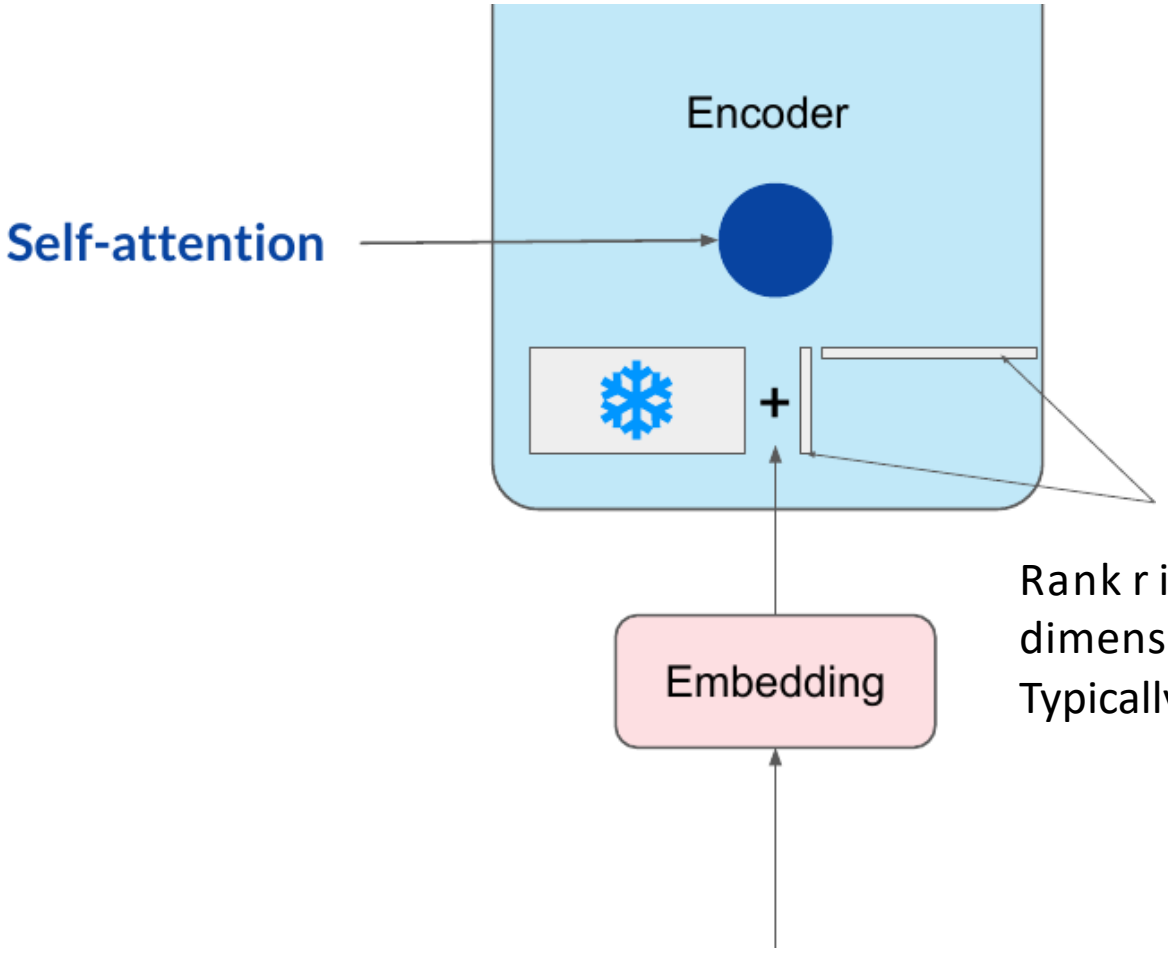


© 2025 TekFrameworks

LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

Step-2: introduce 2 rank decomposition matrices



Rank r is small dimension. Typically, 4, 8 … 64

© 2025 TekFrameworks

LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

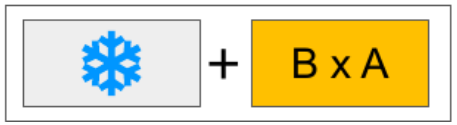
Step-2: introduce 2 rank decomposition matrices

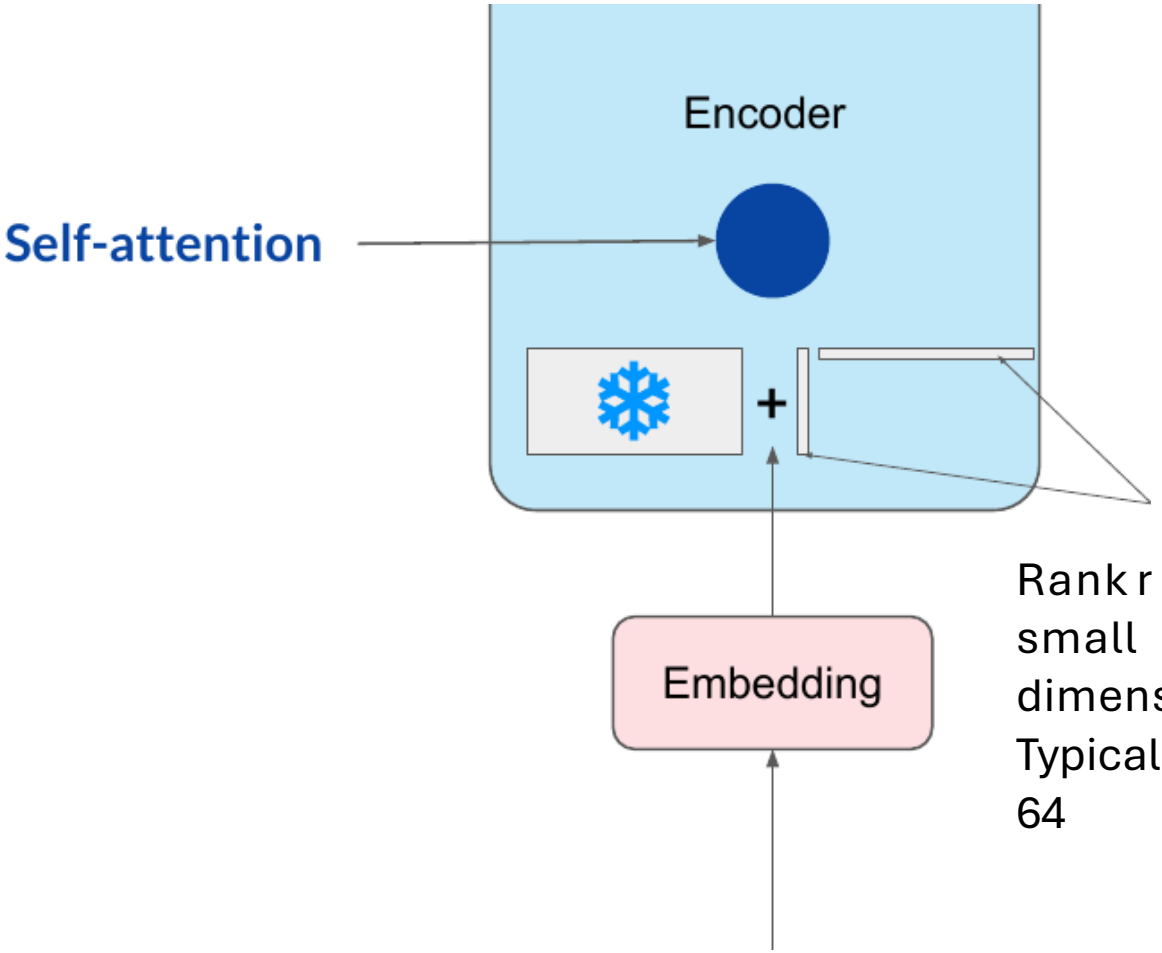
Step-3: Train the weights of the smaller matrices

Steps to update model for inference

1. Matrix multiply the low rank matrices

2. Add to original weights





Rank r is small dimension. Typically, 4, 8 … 64

© 2025 TekFrameworks

LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

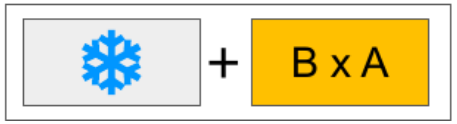
Step-2: introduce 2 rank matrices decomposition .

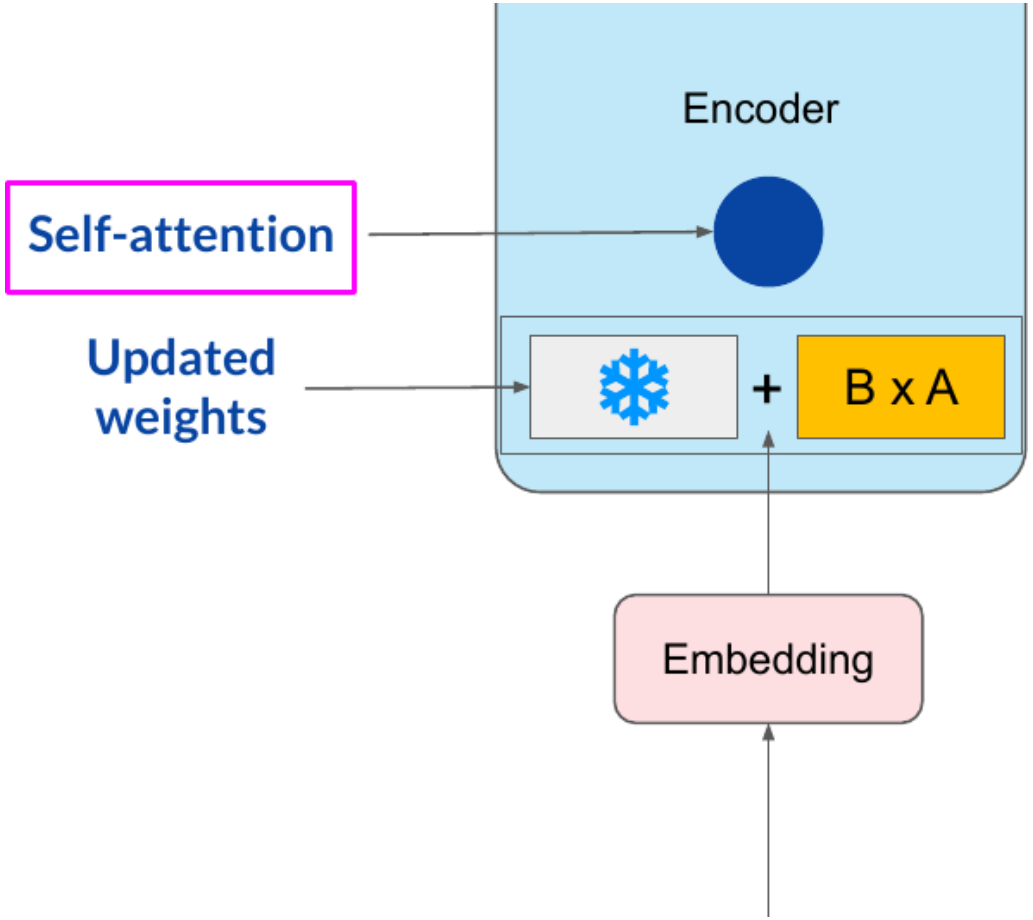
Step-3: Train the weights of the smaller matrices

Steps to update model for inference

1. Matrix multiply the low rank matrices

2. Add to original weights





© 2025 TekFrameworks

Lora: Parameter Reduction Example

- Query matrix from BERT second encoder layer has dimensions d x k = 768 x 768
- So, 768 x 768 = 5,89,824 trainable parameters
- In LoRA with rank r = 32:
- A has dimensions r x k = 32 x 768 = 9216 parameters
- B has dimension d x r = 768 x 32 = 9216 trainable parameters
- 92% reduction in parameters to train!

© 2025 TekFrameworks

LoRA in Practice: What 0.06% Looks Like

```
# Full fine-tuning: ALL weights update
model = AutoModelForCausalLM.from_pretrained("llama-7b")  # 7B params trainable

# LoRA: only A and B matrices train
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"])
model = get_peft_model(model, config)
model.print_trainable_parameters()

# Output: trainable params: 4,194,304 || all params: 6,742,609,920 || trainable%: 0.0622
```

Only 0.06% of weights trained!

`r` sets how much capacity the adapter has
`lora_alpha` sets how loudly it speaks relative to the frozen model
`target_modules` sets where it speaks

© 2025 TekFrameworks

Quantization

© 2025 TekFrameworks

Model Size Reduction

- Most LLMs have billions of parameters → slow, memory-hungry.
- Inference cost is high, especially for cloud and edge deployment.
- Memory constraints limit fine-tuning on smaller GPUs.
- Latency can be a bottleneck for real-time applications.
- Solution: Compress the model without sacrificing much performance.

© 2025 TekFrameworks

Quantization

Quantization helps reduce the memory footprint of a model by storing numbers in 8-bit or 4-bit format instead of 32-bit floats.

For example, a 7B parameter model at FP32 takes ~28 GB; INT4 brings it down to ~3.5–4 GB. The tradeoff is slightly lower accuracy, but often acceptable.

© 2025 TekFrameworks

Quantization: The Math and What It Does to Weights

The Formula

```
q = round(x / scale) + zero_point
```

Simple idea: rescale a float into an integer bucket.

- **`x`** — original float32 weight value
- **`q`** — quantized integer value (INT8 or INT4)
- **`scale`** — step size between quantized levels
- **`zero_point`** — maps float zero to integer range

What It Does to Weights

Each weight (originally 32 bits) is mapped to an integer (8 or 4 bits). The weight matrix looks nearly identical — values shift slightly due to rounding, but the structure is preserved.

Memory impact (7B model):

- FP32 → 28 GB
- INT8 → ~7 GB (4x smaller)
- **INT4 → ~3.5 GB (8x smaller)**

Accuracy loss is slight — often imperceptible in practice. The tradeoff is almost always worth it.

© 2025 TekFrameworks

So Is Quantization A No-brainer?

- At the moment, YES!
- 4x-8x reduction in RAM
- Certain implementations using integer math can be faster by 2-3x on most accelerators
- Use of powerful models on EDGE devices possible

© 2025 TekFrameworks

QLoRA = Quantized Model + LoRA Adapters

- QLoRA uses **4-bit quantized weights** for the base LLM → drastically reduces memory.
- Adds **LoRA adapters** for efficient fine-tuning → no need to update full weights.
- LoRA adapters are trained in **full precision**; only base model is quantized.
- Enables training of models like **LLaMA-7B on a single 24GB GPU**.
- Matches or exceeds full fine-tuning performance — at a fraction of cost and resources.

© 2025 TekFrameworks

QLoRA: Quantized LoRA

- Introduces 4-bit NormalFloat (nf4) data type for 4-bit quantization
- Supports double-quantization to reduce memory ~0.4 bits per parameter
- LoRA adapters at every layer - not just attention layers.

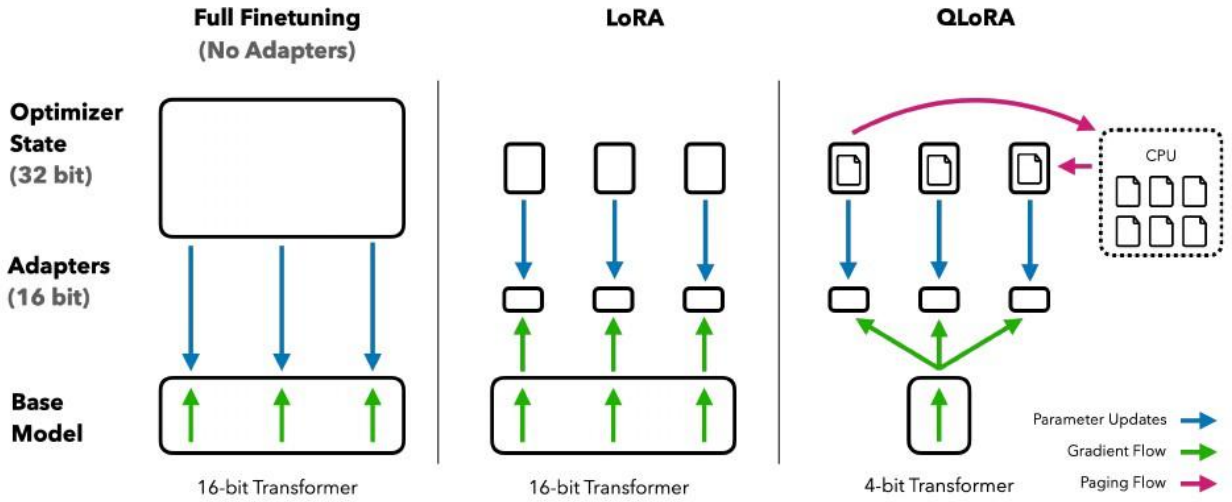


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

© 2025 TekFrameworks

Distillation

© 2025 TekFrameworks

Model Distillation

- A **“student model”** learns to mimic the outputs of a larger **“teacher model”**.
- Transfers knowledge from large models to smaller ones — fewer parameters, faster inference.
- Often used to compress models for real-time or mobile deployment.
- The student model learns from both:
- Ground truth labels
- Teacher’s logits or soft outputs (which carry richer info)
- Can preserve accuracy surprisingly well.

© 2025 TekFrameworks

Does Distillation Violate Scaling Laws?

What Scaling Laws Say

Performance scales with model size, data, and compute — when training from scratch on raw data.

A 1B model trained on raw tokens cannot match a 7B model trained the same way.

So how can a student model match the teacher?

Why Distillation Doesn't Violate Them

Distillation uses a richer training signal: the teacher's probability distributions ("soft outputs").

Example: Teacher assigns 94% Paris, 3% Lyon, 2% Marseille — not just "Paris".

The student learns the **shape of the teacher's uncertainty** — far more information than a one-hot label.

*Geoffrey Hinton called these soft outputs "**dark knowledge**" — the rich structure hiding in the teacher's probability distribution that no single label can capture.*

© 2025 TekFrameworks

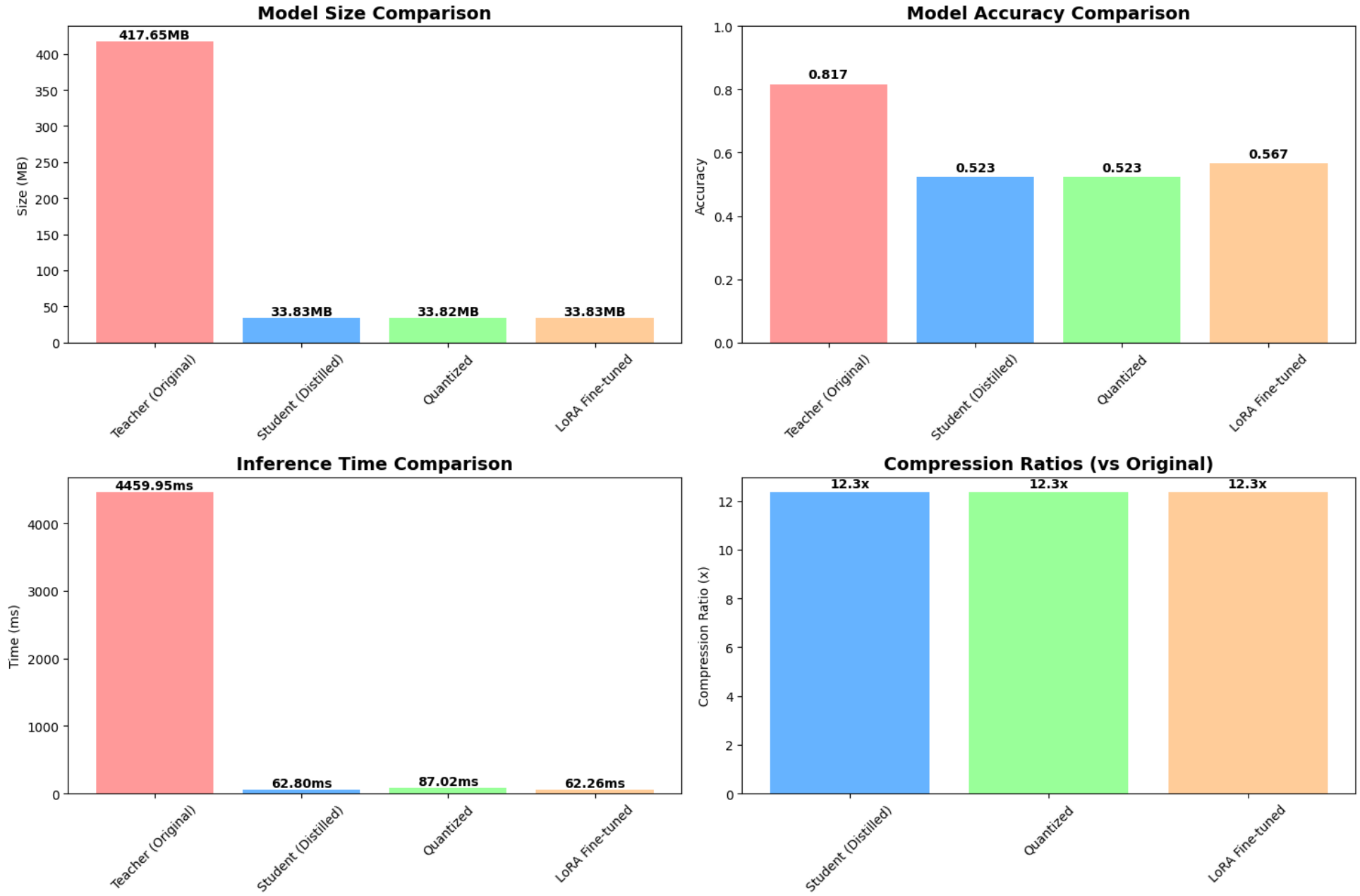
Distilled Models

- Many large models have distilled model twins:
- 30% lesser parameters
- 50% faster
- Retain ~ 95% accuracy
- DistilBERT, DistilROBERTa, DistilGPT-2 are popular distilled models

© 2025 TekFrameworks

Distilled Model Performance

TekFrameworks



DistilBERT (66M) + 8-bit quantization → ~15MB model

Used in **mobile NLP apps** (e.g., **Snips Voice Assistant**) for fast, offline inference with minimal accuracy loss.

© 2025 TekFrameworks

Decision Matrix

When to use what

→

© 2025 TekFrameworks

Distillation vs Quantization

Feature	**Distillation**	**Quantization**
What it does	Trains a small model to mimic a large model	Reduces weight precision (e.g., FP32 → INT4)
Model Architecture	Student model is separate from teacher	Model architecture remains the same
Training Needed	✅ Yes – Student needs to be trained	❌ No (Post-training quantization is common)
Accuracy Impact	Moderate loss (can be minimized)	Slight accuracy drop depending on bits used
Use Case	Create fast, lightweight inference models	Deploy large models on small hardware
Combines With	PEFT, task-specific tuning	LoRA (→ QLoRA), on-device models

© 2025 TekFrameworks

Choosing the right model optimization technique

Scenario / Need	Best Method	Why
Want lowest latency + smallest memory footprint	**Quantization (4-bit / 8-bit)**	Massive RAM reduction, deploy on low-cost GPUs/edge devices
Want small model that behaves like a big model	**Distillation**	Student captures teacher behavior; fast inference
Want to add *new behavior*, style, or output patterns	**PEFT / LoRA**	Efficient weight updates, minimal compute, preserves base LLM
Want to fine-tune *quantized model* on limited GPU (e.g., 24GB)	**QLoRA**	Full finetuning performance on 4-bit model
Need domain-context answers, but not new model behavior	**RAG (next module)**	External knowledge → no model changes
Need full weights updated for highly specialized tasks	**Full Finetuning**	Maximum flexibility, but highest compute/storage cost

Most enterprise tasks can be solved with Quantization + LoRA / QLoRA. Full finetuning is rarely needed today

© 2025 TekFrameworks

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks